Combinational Logic – Part 3

K. Sridharan



Review

- We have discussed decoders.
- We will now discuss about encoders, multiplexers etc.

Encoder

- An encoder performs the converse of a decoder. In essence, we are “coding” the input (and typically have fewer bits at the output).
- That is, we can generate, for example, 3 outputs from 8 inputs. The truth table of an 8-to-3 encoder is shown next.

Truth table of an 8-to-3 encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Realization of the encoder

- The 8-to-3 encoder described can be realized with OR gates. The inputs to these gates are directly obtained from the truth table.
- So we have three OR gates as follows:

$$z = D_1 + D_3 + D_5 + D_7$$
$$y = D_2 + D_3 + D_6 + D_7$$
$$x = D_4 + D_5 + D_6 + D_7$$



Drawback of an encoder

- We have assumed that only one input is 1 at any point of time. That is, only one input is active.
- If two (or more inputs) are active simultaneously, the output will have an undefined combination. For instance, if D_3 and D_6 are 1 simultaneously, the output of the encoder will be 111 (If D_3 is 1, it leads to y and z being 1 while D_6 being 1 corresponds to x and y being 1; see the equations for the OR gates in the previous slide). However, an output of 111 does not represent binary of 3 or 6.

How do we resolve this ?

- To resolve the problem created by simultaneous “on” of two inputs (like D_3 and D_6), we can set a priority.
- That is, we establish an input priority so that only one input is encoded.
- For example, if we assign a higher priority for inputs with higher subscript numbers, even if D_3 and D_6 are 1 simultaneously, the output will be 110 since D_6 has higher priority than D_3 . This leads to what is called as a priority encoder.

Another ambiguity in the output

- An output with all 0's (i.e., 000) is generated when all the inputs are 0 (note that all inputs of 0 are not explicitly considered in the table).
- However, as per the truth table, an output of 000 results when D_0 is alone equal to 1.
- This discrepancy is handled by providing one more output to indicate whether at least one input is equal to 1.

Priority Encoder

- A priority encoder is an encoder circuit with the additional assignment of priority.
- That is, if two or more inputs are equal to 1 at the same time, the input having highest priority will be chosen.
- We show the truth table next for a four-input priority encoder (an encoder as such will have two outputs for 4 inputs but we will add a third output called V whose purpose is to resolve the ambiguity relating to all inputs being 0).

Truth table of a priority encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

Explanation

- Input D_3 has the highest priority so irrespective of the values of the other inputs, when D_3 is 1, the output (for xy) will be 11 (namely, the binary representation for 3).
- Next in the priority order is D_2 and the output is 10 if D_2 = 1 provided D_3 = 0 .
- Note that output will be 10 if D_2 = 1 regardless of the values of the two lower priority inputs (namely D_1 and D_0).

More on the priority encoder table

- With regard to output V (or valid bit), we note that if all inputs are 0, there is no valid input and hence V = 0 (note this additional row in the truth table now). The other two outputs are not inspected when V = 0 and hence are specified as don't care conditions.
- V is set to 1 otherwise (for all other rows) since at least one input has a value of 1.

Expressions for x, y and V

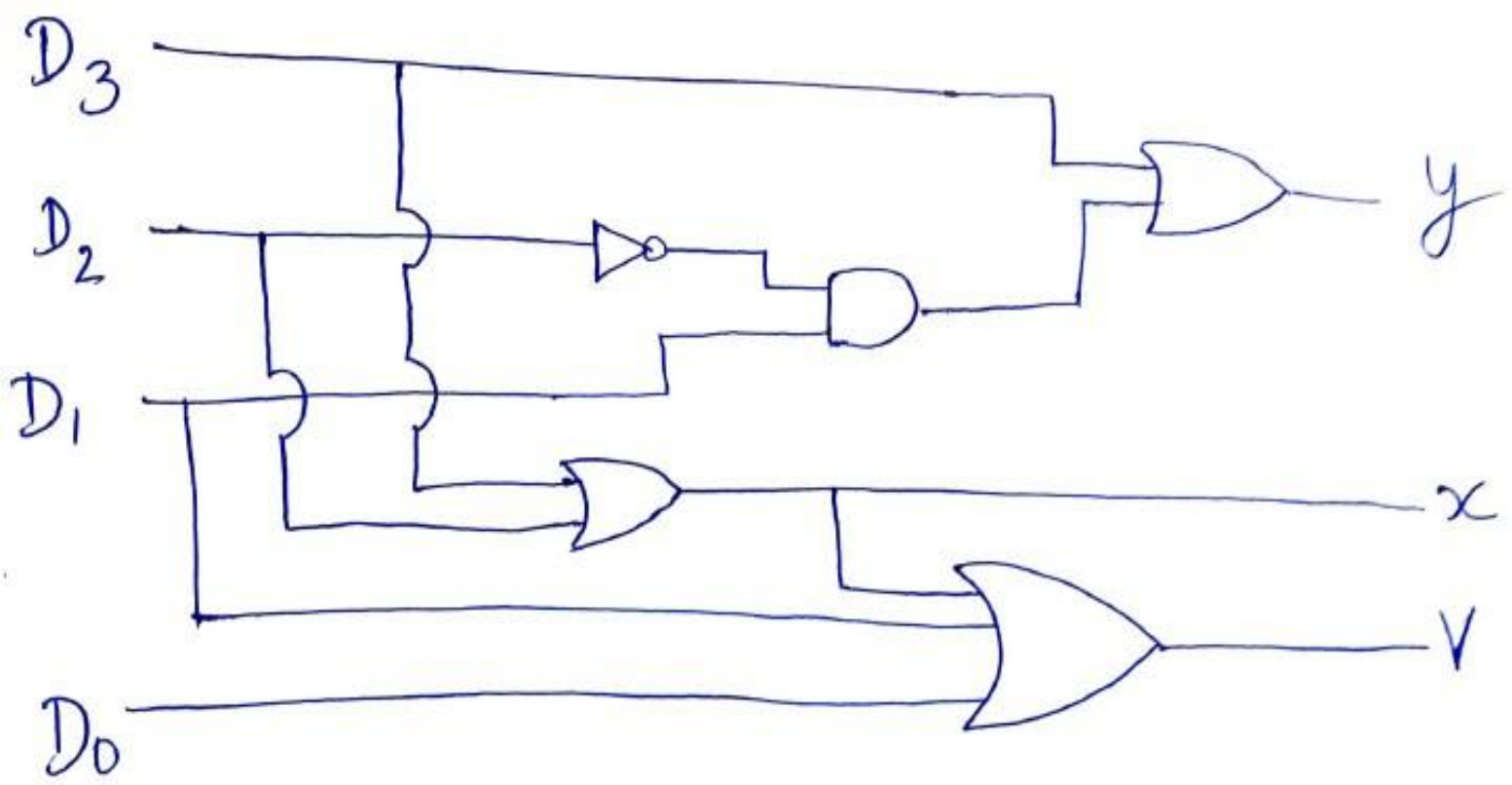
- From the truth table, one can draw Karnaugh maps and get simplified expressions for x, y and V.
- Exercise: Verify the following:

$$x = D_2 + D_3$$

$$y = D_3 + \overline{D_2}\, D_1$$

$$V = D_0 + D_1 + D_2 + D_3$$

Circuit for the priority encoder



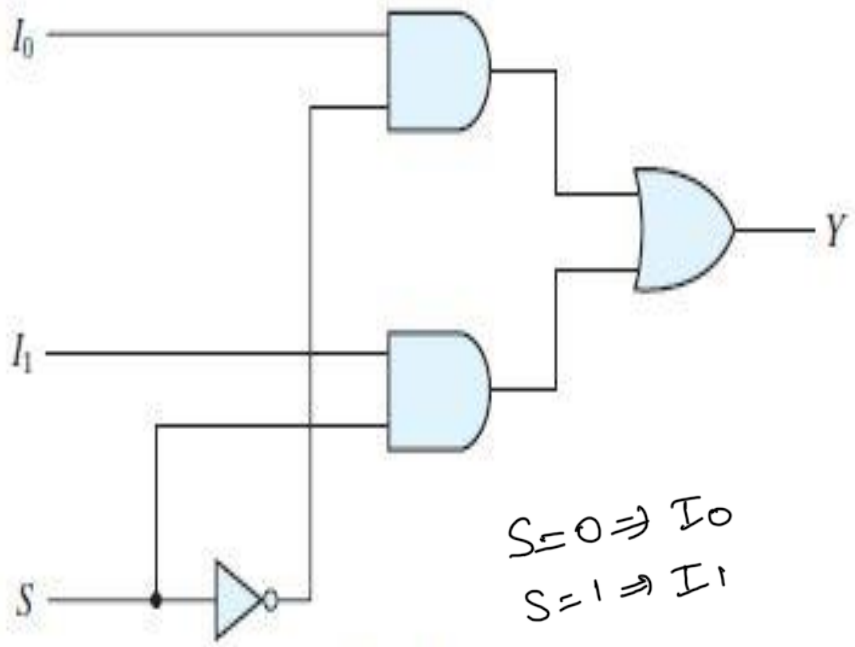
Note on terminology

- The circuit we have just shown is also referred to as a 4-to-2 priority encoder (or a 4-line to 2-line priority encoder)
- The four inputs are D_0 , D_1 , D_2 and D_3 while the 2 outputs are x and y.

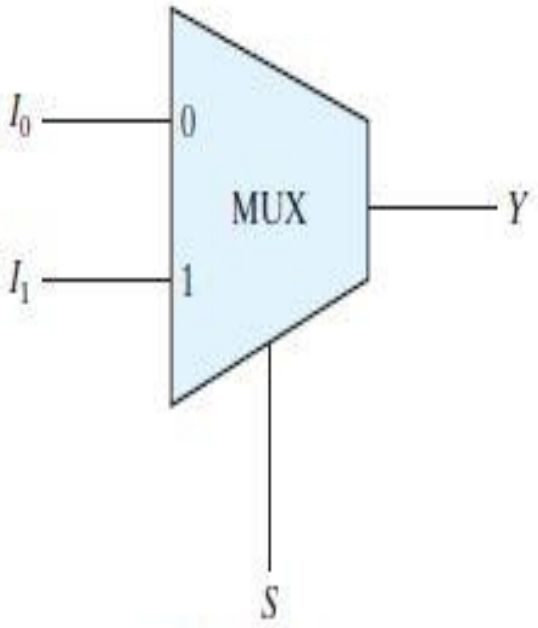
Multiplexer

- Another very useful combinational logic element is a multiplexer.
- A multiplexer selects binary information from one of many input lines and directs to a single output line.
- Multiplexers can be used to realize various Boolean functions.

2-to-1 multiplexer (or 2-to-1 Mux)



(a) Logic diagram

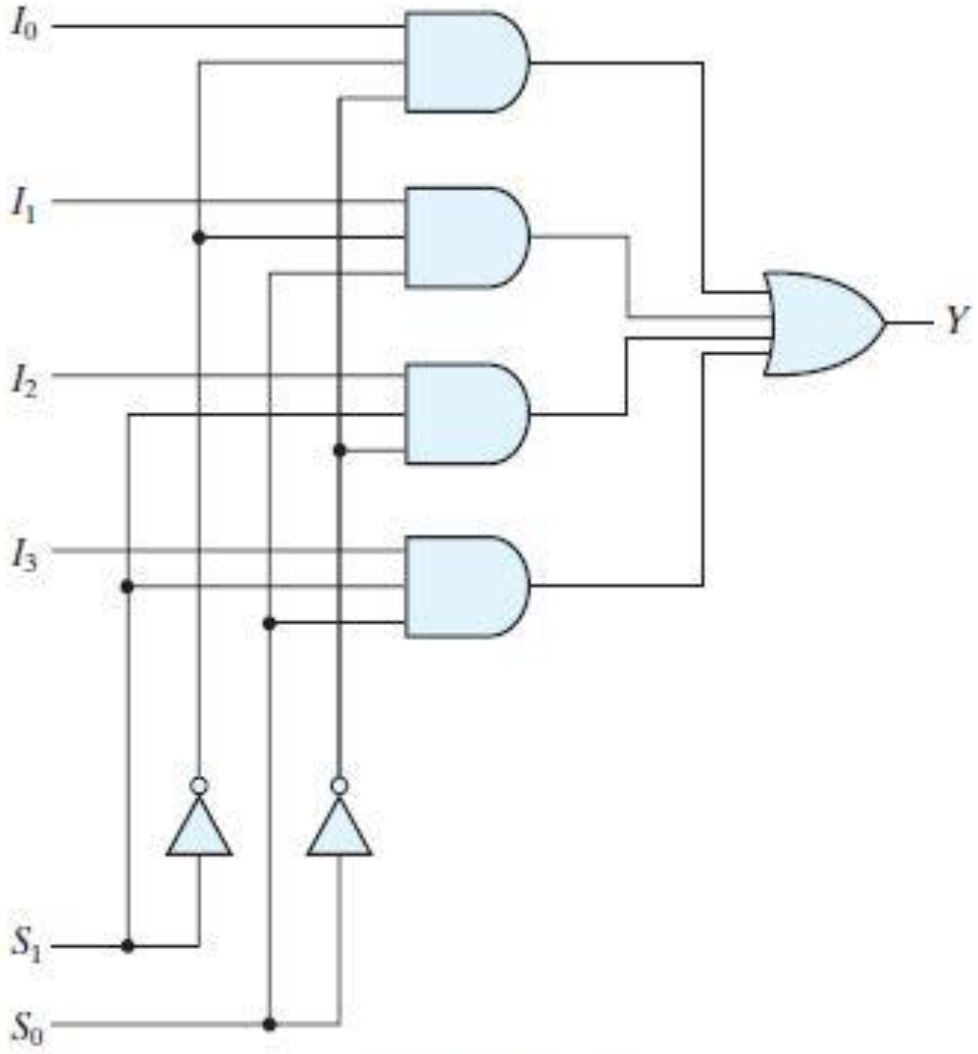


(b) Block diagram

Truth table

- Very straightforward. When S = 0, the output Y will be I_0
- When S = 1, output will be I_1 .
- Can extend to a 4-to-1 mux as shown next.

4-to-1 Mux



(a) Logic diagram

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function table

Utility of a multiplexer

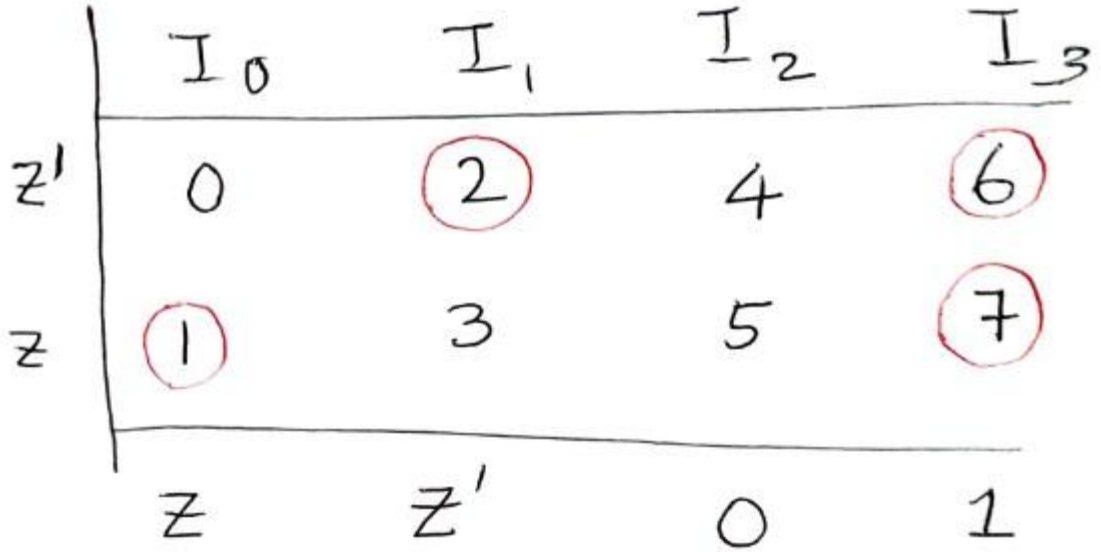
- Recall that a decoder was used along with external OR gates to realize S (sum) and C (carry) of a full-adder.
- Examination of the logic diagram of a multiplexer suggests that it is essentially a decoder that includes the OR gate within the same unit.
- The question is: Can we have a different (and perhaps more efficient) way of using a multiplexer to realize various Boolean functions ?

MUX for implementing Boolean functions

- For realizing a Boolean function of n variables, we can have a multiplexer with n-1 selection inputs.
- The first n-1 variables of the function will be connected to the selection inputs. The remaining variable (in the function) will be used for the data input. If we denote this variable by z, the data inputs of the mux will be z, z', 1 or 0.
- We illustrate this for the function:

  $F(x, y, z) = \sum (1,2,6,7)$

Implementation table for F (x, y, z) = $\sum$ (1,2,6,7)



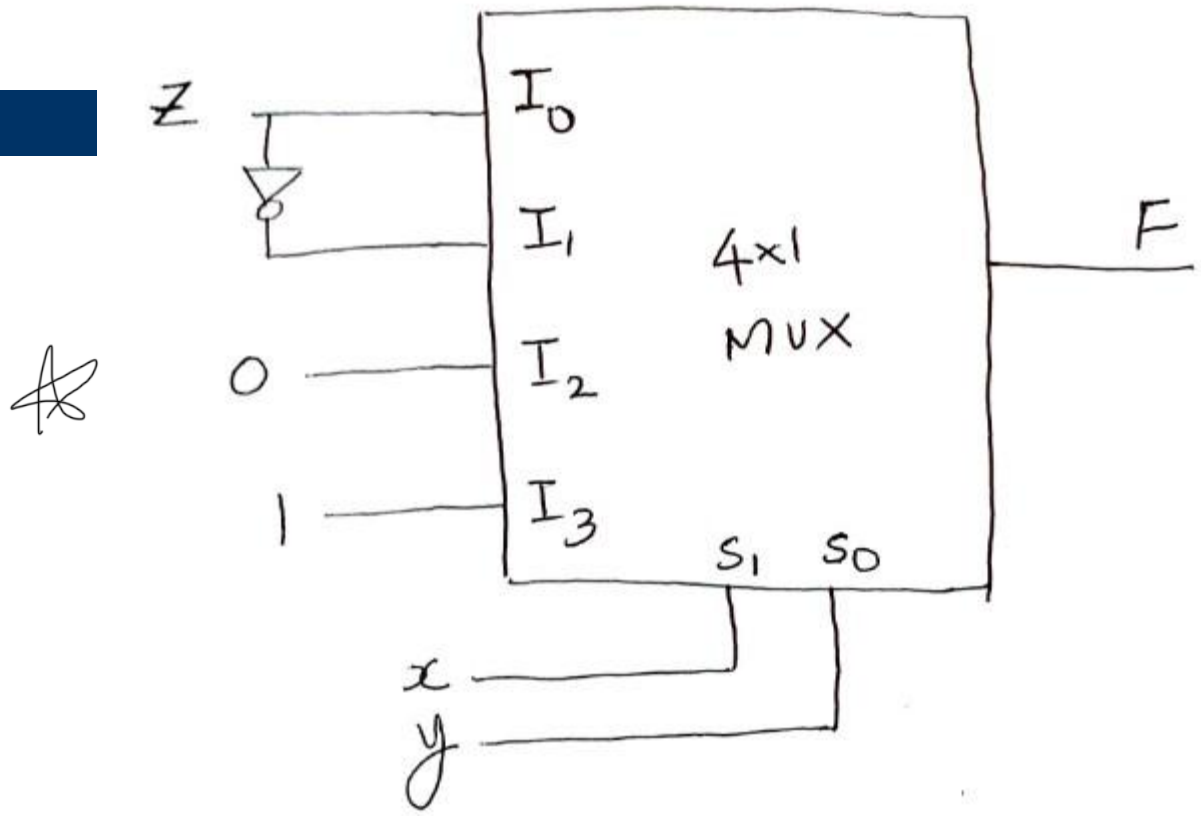
On the implementation table

- Need to pay attention to the way the numbers 0 to 7 are entered here. Thereafter, circle the appropriate minterms.
- If both the minterms in a column are circled, then we apply 1 to the corresponding Mux input.
- If nothing in a column is circled, we apply 0 to the corresponding Mux input.
- If the bottom minterm is only circled, apply z to the corresponding Mux input.
- If the top minterm is alone circled, apply z’ to the corresponding Mux input.

What are constants 1 and 0 ?

- The binary constants 1 and 0 correspond to fixed signal values.
- Logic 0 corresponds to signal ground while logic 1 is the power signal (or V_{cc}).

Mux implementation for F (x, y, z) = $\sum$ (1,2,6,7)



Explanation

- x and y are applied to the selection lines (x is connected to S_1 while y is connected to S_0 input).
- When xy = 00, output F is equal to z since F = 0 when z = 0 and F = 1 when z = 1. This means variable z should be applied to data input I_0.
- Similarly, we can determine the required input to the data lines 1, 2 and 3 from the value of F when xy = 01, 10 and 11 respectively. Will look at more examples in the next part.

Textbook and Reference

- M. Mano and M. Ciletti, Digital Design, Pearson Education, 2018
- C. Roth and L.L. Kinney, Fundamentals of Logic Design, Gamma Publications, 2021